


# Code Metrics

## Mindustry

Realizado por:
Miguel Carmo nº 65871

# # Dependency Metrics Analysis Report

**Project**: Mindustry AI Module

**Metrics Tool:** MetricsReloaded Plugin

**Metrics set:** Dependency Metrics

## 1. Overview

This report analyses Dependency Metrics using the MetricsReloaded plugin to identify coupling issues in the `mindustry.ai` codebase. The analysis covers 43 classes and 4 interfaces across two packages: `mindustry.ai` (core AI) and `mindustry.ai.types` (specialized AI implementations).

## 2. Explanation of Metrics

**Cyclic -** Whether a class participates in cyclic dependency cycles

**Dcy -** Number of dependency cycles a class participates in (higher = more coupled)

**Dpt -** Number of classes this class depends on

**PDcy -** Number of package-level cycles a class participates in

**PDpt -** Number of packages a class depends on (higher = violates package cohesion)

**Impact**: High values indicate tight coupling, making code harder to test and refactor.

**Critical Findings**

- 88.4% of classes (38/43) participate in a large cyclic dependency

- BlockIndexer has the highest dependencies: Dpt=33, PDpt=21

- BaseBuilderAI has the highest cycle participation: Dcy=31

## 3. Trouble Spots

1. **BlockIndexer** (Most Critical)

- Dpt: 33, PDpt: 21

- Code Smell: God Object - too many responsibilities

- Solution: Split into smaller, focused classes

2. **Large Cyclic Dependency**

- 38 classes involved

- Code Smell: Cyclic Dependency

- Solution: Break cycle using dependency inversion or interfaces

3. **Package-Level Cycle**

- Both packages participate in same cycle

- Code Smell: Violation of Acyclic Dependencies Principle

- Solution: Restructure package boundaries

## 4. Discussion: Relation to Code Smells

High Dpt → Code Smells

- Feature Envy: Classes accessing too many external classes (e.g., `BlockIndexer`, `UnitCommand`)

- God Object: Classes with too many responsibilities (e.g., `BlockIndexer` with 33 dependencies)

- Tight Coupling: Hard to test, changes propagate easily

Solution: Reduce dependencies through better encapsulation, split classes, use dependency injection.

## 5. Conclusion

The analysis reveals significant coupling issues:

1. 88.4% of classes participate in a large cyclic dependency
2. BlockIndexer has excessive dependencies (Dpt=33, PDpt=21)
3. Both packages participate in the same cycle, violating package principles
4. Code smells identified: God Object, Feature Envy, Cyclic Dependency, Rigid Design

Regular monitoring of these metrics will help prevent future coupling issues and improve code maintainability.